

INT 161

Basic Backend Development

Week-03

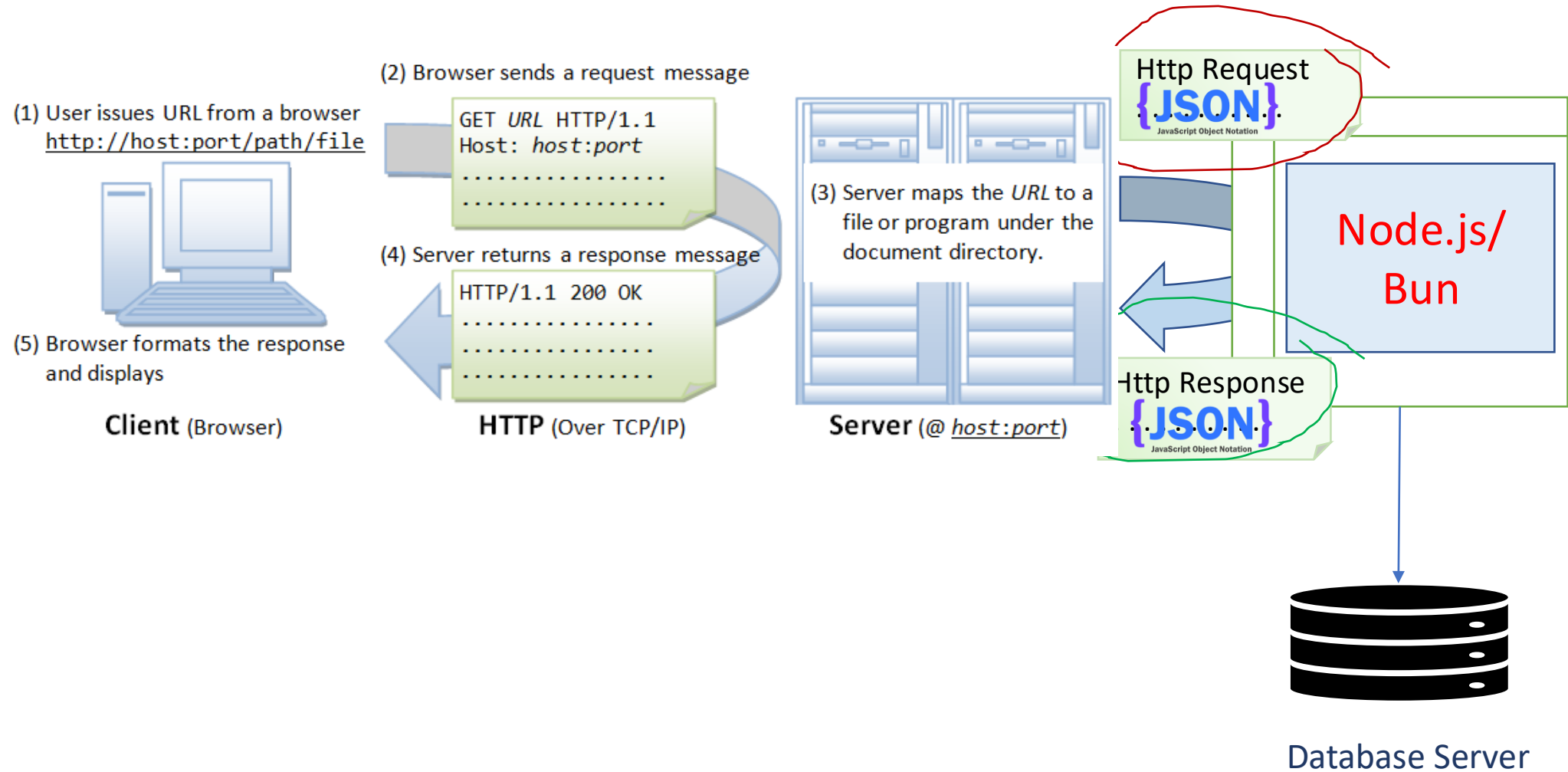


Introduction to RESTful API

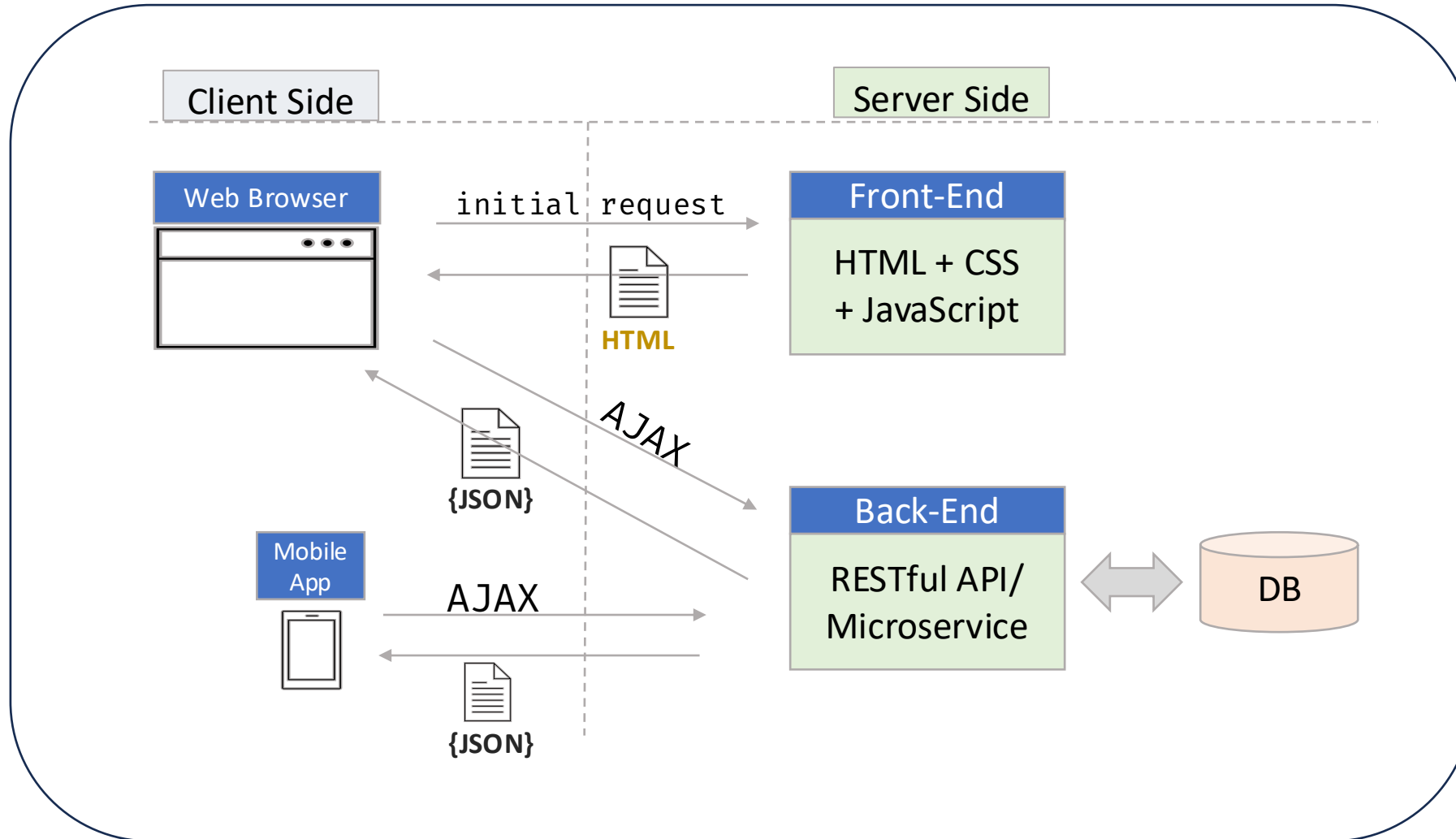
Agenda

- REST Principles Overview
- Deep Dive: Uniform Interface
- Deep Dive: Layered System
- Implementing REST without Frameworks (Built-in http Module)
- Building a Layered Architecture in Pure Node.js
- Key Takeaways

Web Application



SPA – Single Page Application architectures



REST API (also known as RESTful API)

- REST stands for **RE**presentational **S**tate **T**ransfer and was created by computer scientist Roy Fielding.
- An application programming interface (API or web API) that conforms to the **constraints** of **REST architectural style** and allows for interaction with RESTful web services.
- In REST architecture, a REST Server simply provides access to resources and REST client accesses and modifies the **resources**.
- **Each resource is identified by URIs/ global IDs.**
- REST uses various representation to represent a resource like text, JSON, XML. JSON is the most popular one.

<https://restfulapi.net/>

https://www.w3schools.com/nodejs/nodejs_rest_api.asp

RESTful Principles and Constraints

- RESTful Client-Server

- Server will have a RESTful web service which would provide the required functionality to the client.
- The client send's a request to the web service on the server. The server would either reject the request or comply and provide an adequate response to the client.

- Stateless

- Statelessness mandates that each request from the client to the server must contain all of the information necessary to understand and complete the request.
- The server cannot take advantage of any previously stored context information on the server.
- For this reason, the **client application** must entirely keep the session state.

<https://restfulapi.net/>

RESTful Principles and Constraints (2)

- Interface/Uniform **Contract**

- This is the underlying technique of how RESTful web services should work. RESTful basically works on the HTTP web layer and uses the below key verbs to work with resources on the server.
 - **POST** – To create a resource on the server
 - **GET** – To retrieve a resource from the server
 - **PUT** – To change the state of a resource or to update it
 - **DELETE** – To remove or delete a resource from the server

- Layered system

- The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior.
- For example, in a layered system, each component cannot see beyond the immediate layer they are interacting with.

RESTful Principles and Constraints (3)

- Cacheable

- The cacheable constraint requires that a response should implicitly or explicitly label itself as cacheable or non-cacheable.
- If the response is cacheable, the client application gets the right to reuse the response data later for equivalent requests and a specified period.

- Code on demand (optional)

- REST also allows client functionality to extend by downloading and executing code in the form of applets or scripts.
- Servers can provide part of features delivered to the client in the form of code, and the client only needs to execute the code.

Uniform Interface

- The fundamental constraint that decouples clients from server implementations
- 4 Core Pillars:
 - Resource Identification: URIs define resources, **not actions** (e.g., [/api/v1/users](#))
 - Manipulation via Representations: Exchange state using standard formats (JSON)
 - Self-Descriptive Messages: Use standard HTTP methods ([GET](#), [POST](#), [PUT](#), [DELETE](#)) and headers (Content-Type: application/json)
 - HATEOAS: Hypermedia links guide client actions dynamically

<https://restfulapi.net/resource-naming/>

HTTP Methods

- **Method Purpose Example**

- **GET** Retrieve data `/api/v1/users`
- **POST** Create new resource `/api/v1/users (with data)`
- **PUT** Update existing data `/ api/v1/users/{id} (with data)`
- **DELETE** Remove a resource `/ api/v1/users/{id}`

Rules of REST API endpoints.

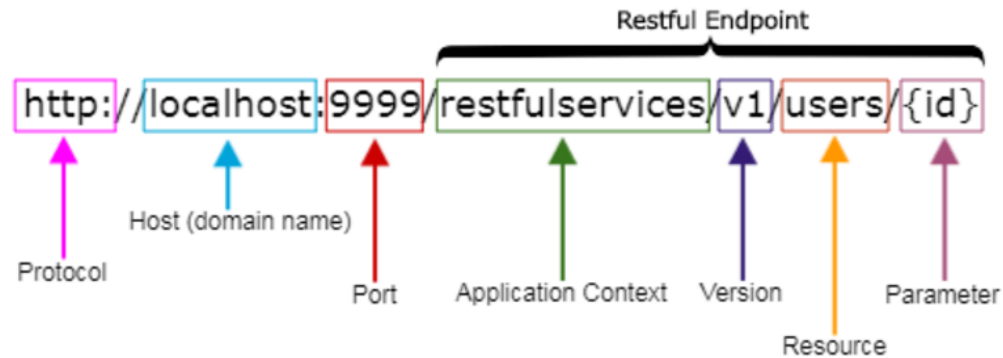
- There are certain rules which should be kept in mind while creating REST **API endpoints**.
 - REST is based on the resource or **noun instead of action or verb based**. It means that a URI of a REST API should always end with a noun. Example: **/api/users** is a good example.
 - HTTP verbs are used to identify the action. Some of the HTTP verbs are – **GET**, **PUT**, **POST**, **DELETE**, **PATCH**.
 - A web application should be organized into resources like users and then uses HTTP verbs like – GET, PUT, POST, DELETE to modify those resources. And as a developer it should be clear that what needs to be done just by **looking at the endpoint and HTTP method used**.

RESTful Resource Uniform Interface

URI	HTTP Verb	Description
api/ products	GET	Get all products
api/ products /1	GET	Get an products with id = 1
api/ products /1/ orders	GET	Get all orders for product id = 1
api/ products	POST	Add new products
api/ products /1	PUT	Update an products with id = 1
api/ products /1	DELETE	Delete an products with id = 1

Always use **plurals** in URL to keep an API URI consistent throughout the application.
Send a proper HTTP code to indicate a success or error status.

REST API URI Naming Conventions and Best Practices



- Singleton and Collection Resources

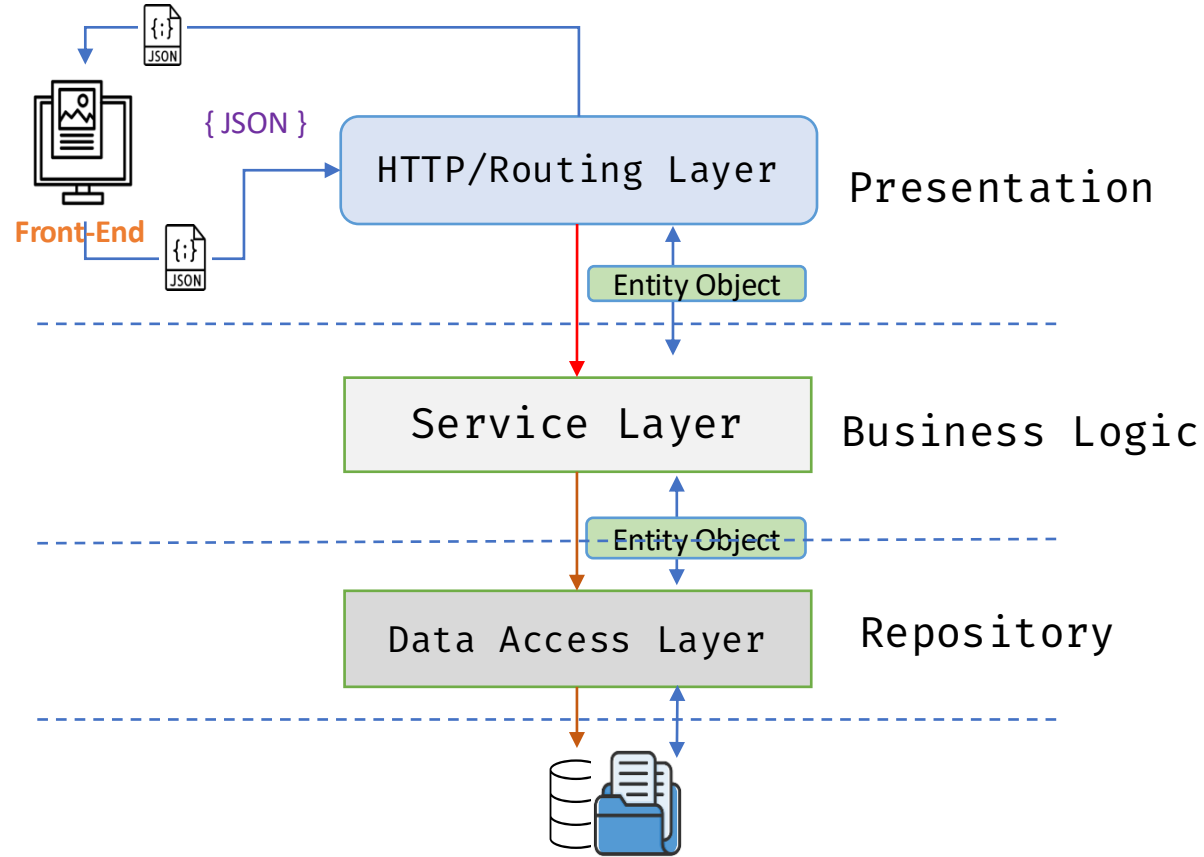
<code>/customers</code>	// is a collection resource
<code>/customers/{id}</code>	// is a singleton resource

- Sub-collection Resources

<code>/customers/{id}/orders</code>	// is a sub-collection resource
-------------------------------------	---------------------------------

<https://restfulapi.net/resource-naming>

Layered System



Concept: Hierarchy of intermediate layers where each layer only interacts with its adjacent neighbor

Benefits:

- **Scalability:** Shared caches and load balancers can be added without client changes
- **Separation of Concerns:** Keep raw HTTP parsing completely isolated from core business logic and database interactions
- **Security**

Designing Software Layers in Pure Node.js

- Separate concerns cleanly without external frameworks:

```
src/  
├── server.js      --> Native HTTP Server Setup  
├── router.js     --> Route Matching & HTTP Handling  
├── services/     --> Business Logic (Framework-Agnostic)  
└── repositories/ --> Data Access Layer (File System / DB)
```

Practice

